

A Project report on

Fire Extinguishing Robot and Navigating System using Arduino UNO

Submitted for partial fulfillment of Engineering Explorations Course
For Bachelor of Technology
Degree in
Computer Science and Engineering
For the academic year 2023-24
By,

Sl	USN	Name
1	1RVU23CSE070	ANUSREE KRISHNAN
2	1RVU23CSE003	AADYA ARUN TALREJA
3	1RVU23CSE073	ARCHITA DAS
4	1RVU23CSE075	ARJUN SYAM
5	1RVU23CSE269	MISHA PHOKELA
6	1RVU23CSE273	MITUL K M

Title of the Domain: **Robotics**
Name of the Domain Lead: **Professor Thotreithem Hongray**



School of Computer Science and Engineering
RV University
RV Vidyanikethan Post, 8th Mile, Mysore Rd, Mailasandra,
Bengaluru, Karnataka 560059



**RV
UNIVERSITY**

Go, change the world

an initiative of RV EDUCATIONAL INSTITUTIONS

CERTIFICATE

This is to Certify that the project work “**Fire Extinguishing and Navigating System using Arduino UNO**” carried out by Aadya Arun Talreja (1RVU23CSE003), Anusree Krishnan (1RVU23CSE070), Archita Das (1RVU23CSE073), Arjun Syam (1RVU23CSE075), Misha Phokela (1RVU23CSE269), Mitul KM (1RVU23CSE273) bonafide students of SoCSE in partial fulfillment of the Engineering Explorations course of Bachelor of Technology in Computer Science and Engineering of the RV, Bengaluru, during the academic year 2023-24. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in this report. The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said course.

Title of the Domain: **Robotics**

Signature of the Domain Lead

Name of the Domain Lead: **Professor Thotreithem Hongray**

Fire Extinguishing and Navigating System using an Arduino UNO

Acknowledgement

We would like to express our utmost gratitude and special thanks to Professor Hongray, our domain lead and project supervisor, for guiding us through the process of completing this project. He has given us lots of guidance and constructive advice on how to resolve all the problems, conundrums, and issues that arise along the way in completing this project. He has shared his precious knowledge and experience accumulated over the years in guiding through and supervising his students in all kinds of projects related to the Robotics field, ranging from circuit constructions, microcontroller C programming techniques, and documentation work such as the technical paper and the final report. He provided us with necessary technical solutions, advice and guidance to aid us in overcoming the issues faced.

We would like to thank Harsha Sir, Baishali Ma'am, Amruthesh Sir, Durgarao Sir and the whole of the Engineering Exploration and Exploring Science-2 team for giving us this opportunity to work on this project.

We would like to thank all of our coursemates and friends, who have directly or indirectly contributed to our project. We would like to express our gratitude to Team 4: Bhumika G, Bhuvan Madhusudhan, Anirudh M, Aniketh Chakravarthy, Achyuth Anil Kumar, Sujay S Veershetta. Their contributions aided us in completing the prototype in the best way possible. We would like to take this opportunity to thank our family members, friends and lecturers for their continuous support and encouragement given to us.

Table of Contents

Sl no.	Title	Page Number
1.	Introduction	5-6
	(i) Purpose	5
	(ii) Motive	5
	(iii) Scope of the project	5
	(iv) Literature Survey	6
2.	Abstract	7
3.	Material and Components Required	8-13
	(a) Arduino UNO	8
	(b) Motor Shield	9
	(c) Single Channel Relay	10
	(d) Flame IR Sensor	11
	(e) Heat Sensor	12
	(f) Servo Motor	13
4.	Circuit Diagram and Flow Chart	14-16
	(i) Schematic Diagram	14
	(ii) Flowchart of Hardware Working	15
	(iii) Flowchart of Code Working	16
5.	Data Analysis and Acquisition	17-19
	(i) Robot Response and Arduino Uno Action	17
	(ii) Heat (Temperature) vs. Pump Status	17
	(iii) Variation of Motor Speed over Time	18
	(iv) Temperature vs. Flame IR	18
	(v) Robot Movement vs. Flame IR	19

Sl no.	Title	Page Number
6.	Data Visualization	20-24
	(i) Pump Status vs Heat (Temperature)	20
	(ii) Flame IR Sensor vs Temperature	20
	(iii) IR Sensor Reading for Each Temperature	21
	(iv) IR Sensor Readings Over Temperature	21
	(v) Motor Speed Over Time	22
	(vi) Wheel Speed over Time	22
	(vii) Distribution of Motor and Wheel Speeds	23
	(viii) Temperature Over Time	24
	(ix) Activation of IR Sensors Over Time	24
7.	Discussion and Conclusion	25-27
	(a) Problems faced	25
	(b) Solutions	26
	(c) Application	26
	(d) Future work	26
	(e) Future Scope	27
	(f) Advantages & Disadvantages	27
8.	References	28
9.	Appendix	29-32

Introduction

Purpose

A fire extinguishing robot is designed to enter dangerous situations and extinguish fires while minimising the risk to human firefighters. These robots, which can be controlled remotely from a distance, employ sensors to gather real-time data which can be used by the firefighters to efficiently put out the fire. The robots can explore a large variety of terrain and areas with limited access using the resources wisely and reduce the overall damage. The main and only work is to deploy the robot in a fire prone area and the robot will automatically work once it detects a fire breakout. This prototype helps in Rescue operations during fire accidents where the entry of service man is very difficult in the fire prone area. There are several existing types of vehicles for firefighting at home and extinguish forest fires. Our proposed robot is designed to be able to work on its own or be controlled remotely. By using such robots, fire identification and rescue activities can be done with higher security without placing fire fighters at high risk and dangerous conditions. In other words, robots can reduce the need for firefighters to get into dangerous situations.

Motive

The project is motivated by the urgent need to better improve firefighter safety and effectiveness. Fire extinguishing robots autonomously or via remote control, enter hazardous environments, reducing risk to human firefighters. The robots provide real-time data and precise control, improving emergency response. Fire incident is a disaster that can potentially cause the loss of life, property damage and permanent disability to the affected victim. Fire fighters are primarily tasked to handle fire incidents, but they are often exposed to higher risks when extinguishing fire, especially in hazardous environments such as in nuclear power plant, petroleum refineries and gas tanks. They are also faced with other difficulties, particularly if fire occurs in narrow and restricted places, as it is necessary to explore the ruins of buildings and obstacles to extinguish the fire and save the victim. Additionally, the project aims to minimise response times, ultimately saving lives and protecting property.

Scope of the project

Our fire extinguishing robot, primarily makes use of infrared (IR) sensors to detect any fires in the surrounding successfully. These sensors are calibrated to detect specific heat signatures, allowing the robot to detect potential fires in its vicinity. Our fire extinguishing robot operates with a simple yet effective mechanism: it moves forward until its infrared sensors detect a fire, at which point it promptly stops. This approach ensures safety of the robot, allowing it to effectively locate and suppress fires as quickly as possible.

Literature Survey

The primary objective of this project is to create and deploy an Autonomous Fire-Fighting Mobile Platform capable of monitoring as well as detecting the presence, locating the exact position of the fire, extinguishing the flame and returning safely. This project's top priority is the ability to detect fires accurately.

Overall, the major goal of this project is for the prototype to carry out its intended role with minimal faults. The development of a fire-fighting robot was influenced by a variety of sources, including internet instructions and tutorial videos.

One of our key sources is Techatronic.com's article "Fire Fighter Robot using Arduino," which has thorough instructions for creating a firefighting robot with an Arduino. The basis for determining the essential elements and features required to execute a project was based on this data. In addition, the development process was assisted by YouTube videos demonstrating how this project works. In order to assist in the practical implementation of the project, the videos included visual demonstrations and step-by-step instructions.

These websites had great information on programming and sensor integration, even though it isn't specifically related to combating fires. The team's diverse expertise allowed us to gain a comprehensive understanding of the concepts and procedures needed for the design and functioning of fire extinguisher robots.

We have used:

1. <https://techatronic.com/fire-fighter-robot-using-arduino-fire-fighting-robot/> ,
2. <https://www.youtube.com/watch?v=bRZNyQtwZVE>
3. <https://www.instructables.com/Obstacle-Avoiding-Robot-Arduino-1/>

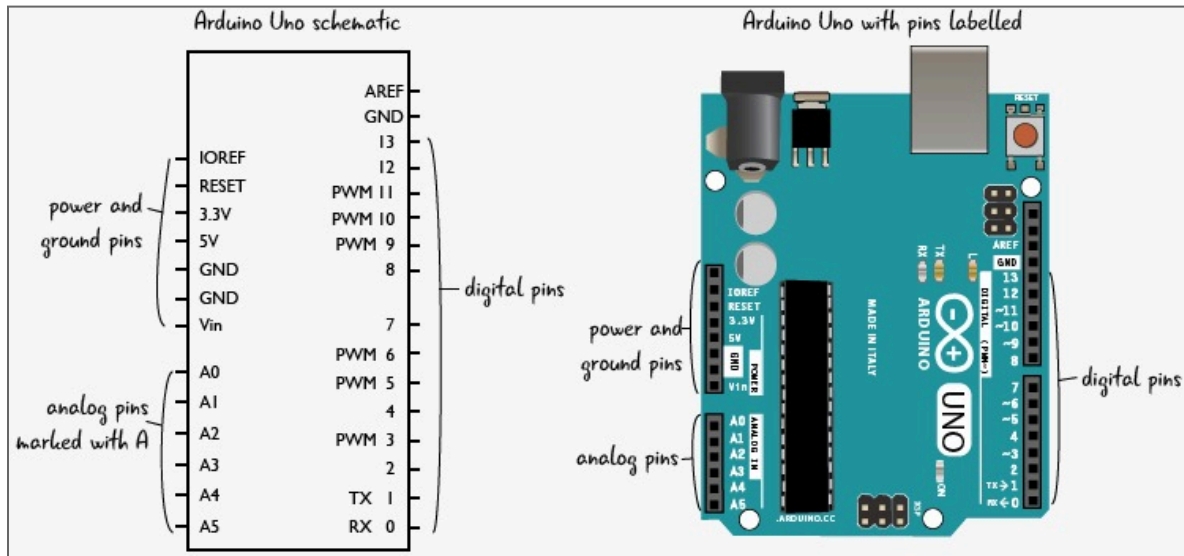
as our main guide to build our robot.

Abstract

The project aims to develop and deploy a fire-extinguishing robot equipped with infrared (IR) sensors for fire detection and suppression in hazardous environments. Key features include sensor integration, fire detection and suppression. The objective is to enhance firefighting efficiency and safety by providing firefighters with a tool for swift intervention in emergencies. By putting a mobile robot to perform this task in a fire prone area, it can help to avoid untoward incidents or the loss of lives. This project is enhanced to control fire through a robotic vehicle. With the advancement in the field of Robotics, human intervention is becoming less every day and robots are used widely for the purpose of safety. In our day to day life fire accidents are very common and sometimes it becomes very difficult for firemen to save human life. In such cases firefighting robots come into picture.

Material and Components Required

1. Arduino UNO



The Arduino UNO is a microcontroller-based board that provides a low-cost, flexible, and easy-to-use programmable board with an ability to be integrated within various electronic projects. The microcontroller that powers the system is that of ATmega328P which is a high performance 8-bit AVR RISC-based microcontroller.

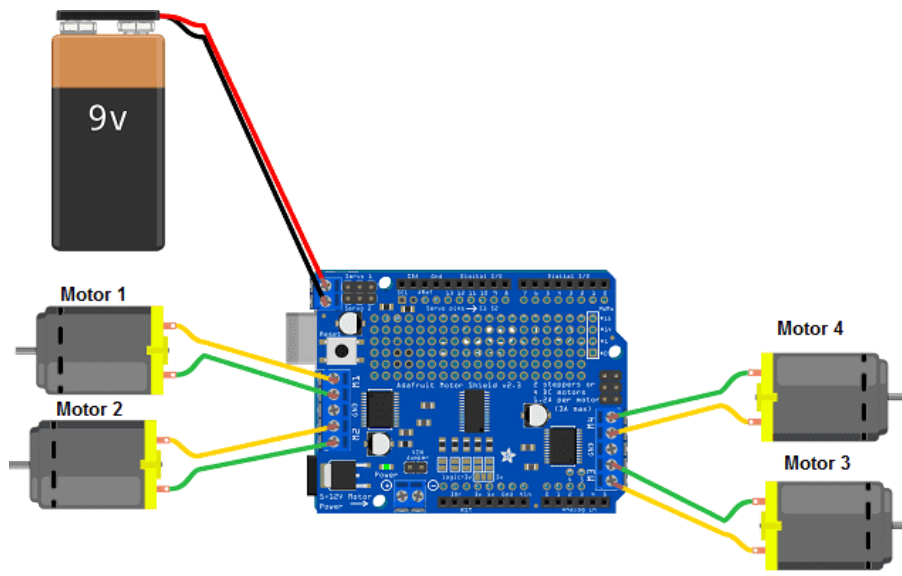
Arduino specifications:

1. Microcontroller: ATmega328P (8-bit, RISC architecture)
2. Analog Inputs: 6
3. Digital I/O pins: 14 (6 PWM outputs)
4. Memory:
 - a. 32 KB Flash
 - b. 2KB SRAM
 - c. 1 KB EEPROM
5. Clock speed: 16 MHz
6. Communication: USB, UART, I2C, SPI

Programs uploaded into the flash memory within the Arduino UNO facilitate communication between the board and electronic components. This enables users to control and tune the working of the electronic components to their desire. The programs are uploaded into the flash memory over USB. The microcontroller clocked at 16 MHz, then efficiently executes the program that is uploaded effectively. The Analog inputs provide the ability to read Analog inputs from specific components. Along with this, the Digital I/O pins allow for reading and writing digital signals to and from the components and The board. The communication protocols consisting of USB, UART, I2C, SPI all enable for communication of the Arduino UNO with other devices.

Overall Uno enables us to create complex and robust projects by comprehending and executing user-created programs to control various subcomponents and formulate them into a single robust system.

2. Motor Shield

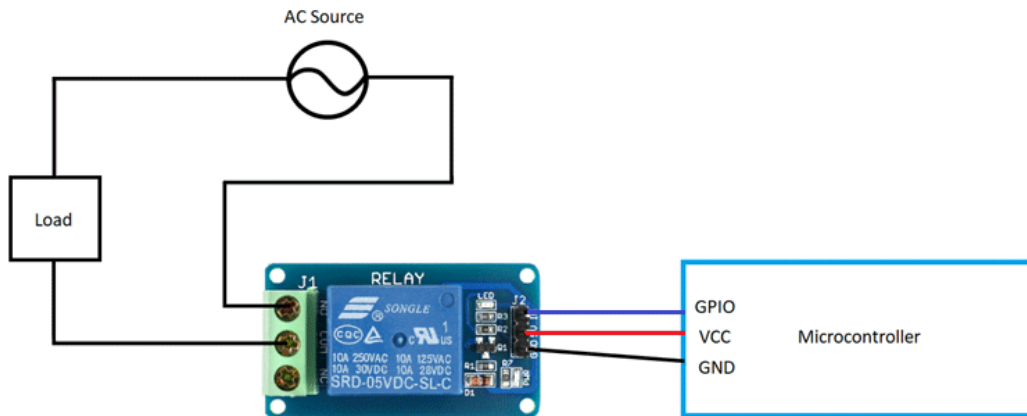


The L293D Motor Shield is a highly versatile driver module that enables users to control the working speed and direction of 4 motors along with 2 servo motors. This Arduino compatible motor driver can be used to drive 4 DC motors or 4 wire steppers and 2 servos. It drives the motors with external power input.

The shield also includes 6 5v VCC and Ground ports along with additional access to the 6 Analog pins of the Arduino. The L293D drives the DC motors and steppers while Arduino pins 9 and 10 drive the servo motors. This particular motor shield is based on the L293 IC. The IC enables control over 2 DC motors in any direction. Thus the shield includes 2 of such ICs along with one 74HC595 shift register.

The shift register expands 3 pins of the Arduino to 8 pins to control the direction of the motors. The outputs given by the L293D are directly connected to the PWM outputs of the Arduino.

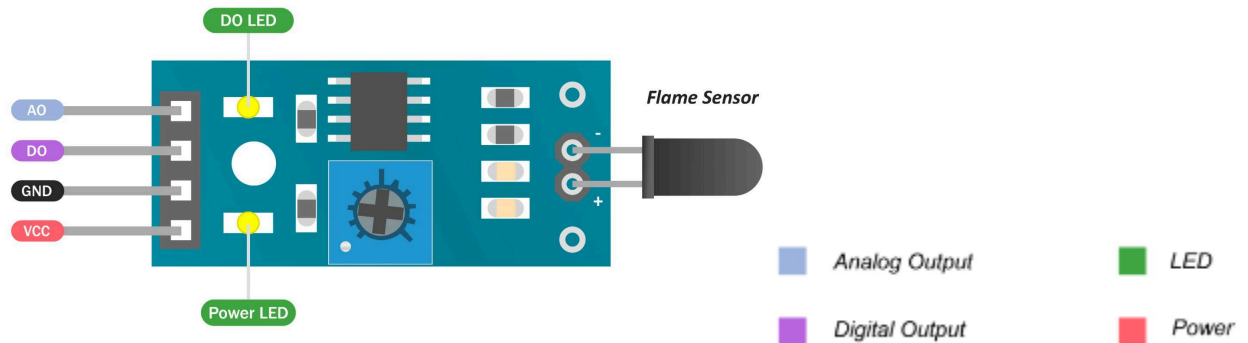
3. Single Channel Relay



The Signal Channel Relay is a convenient module that allows users to control a singular high voltage, high current load within a circuit. Using this module we can control high-voltage or high-power devices. The modules pinout include digital input(IN), GND, VCC, NO(Normally Open), COM (Common), and NC(Normally Closed).

The relay acts like a switch. It consists of an Armature that is directly connected to the common port. The armature is always in contact with the NC closing the loop between NC and COM. When current is applied to an electromagnet built within the relay it moves the armature and enables contact with the NO port thus closing the loop between the NO and the com. Thus, the relay acts as a remote-controlled switch disabling or sending current to a particular device.

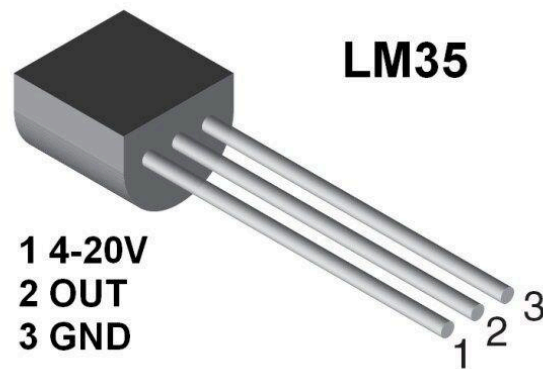
4. Flame IR Sensor



In this regard, a flame infrared sensor is an exceedingly advanced device aimed at identifying the distinctive infrared radiation emitted by the flames during the process of combustion. While human vision utilises the set of wavelengths labelled as evident light, flames also radiate infrared wavelengths that are usually invisible for the naked eye. Use structured light to spot this exception in the vicinity, discarding different probable sources of light as background noise. For that reason, an infrared (IR) sensor is being used; a tiny video camera that is made solely for detecting IR light. This features an optical sensor configured to get rid of any possible source of light so that it can receive accurate data from different conditions. A signal processing device supports these tools which interpret the data gathered and identify relevant characteristics of flame. Flames are highly radiant in the infrared region with emission over a wide band of wavelengths, usually between 4-10 μm .

Moreover, it can detect shorter wavelengths emitted by flames between 1.0 and 2.7 micrometres, thereby increasing its usefulness in fire detection. Most importantly, the sensor can distinguish authentic fire emissions from ambient IR sources by their flames' distinctive flickering patterns. This characteristic is very important in critical applications to avoid false alarms and ensure high instrument reliability. When installed, it delivers a panoramic field of view up to 60 degrees and detects within a range of 100 cm. This means that wherever it is mounted, the sensor covers a wide area. In fact, an infrared flame sensor operates much like a specialised camera that is employed for sensing invisible infrared emissions from fires. Through a degree of tuning and flexibility, it enhances fire suppression capabilities significantly.

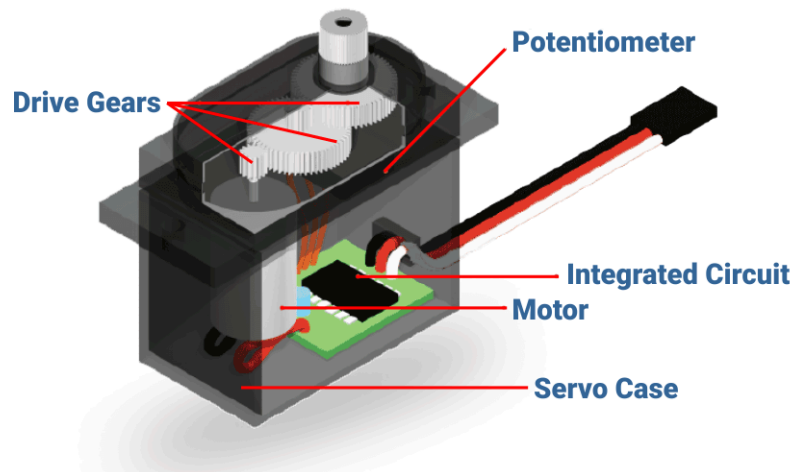
5. Heat Sensor



The LM35 behaves much like a diode, with its voltage linearly increasing as the temperature rises or falls. It has a precision integrated circuit known as LM35 which contains this kind of average type of IC within it. For instance, there is a substrate with integrated resistors inside it, and a transistor for that matter. It measures temperatures precisely because the thermometric features of the substrate and transistor change accordingly with varying degrees on scale being measured by means of LM 35 itself in reality. To work, a heat sensor detects the temperature in its environment and becomes active when it reaches a certain threshold. Just like temperature sensors, these sensors are solid-state designed to make sure that they are accurate and durable without any mechanical component.

A heat sensor becomes operational in its environment by detecting the temperature, and activates when it reaches a certain threshold. These sensors are solid-state like temperature sensors and they do not have any mechanical part to ensure accuracy and durability.

6. Servo Motor



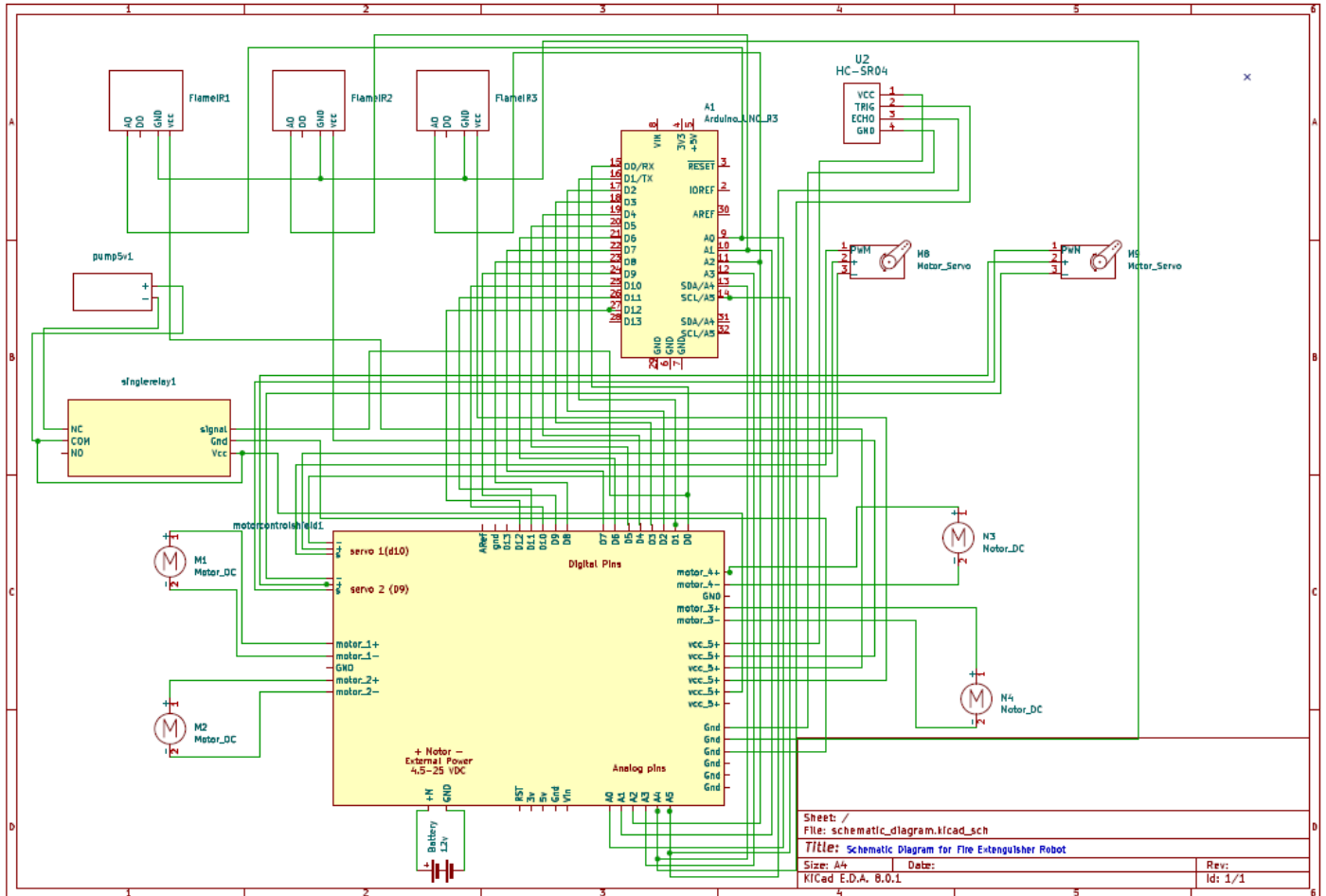
A servo motor is a versatile device commonly used in robotics and automation for precise control of angular or linear motion. It consists of several key components that work together to achieve accurate positioning and movement.

1. **Potentiometer:** The potentiometer is a variable resistor that serves as feedback for the motor's current position. It provides positional information to the servo controller, allowing for closed-loop control and accurate positioning.
2. **Drive Gears:** The drive gears transmit rotational motion from the motor to the output shaft, amplifying torque and controlling speed. These gears play a crucial role in converting the motor's rotary motion into linear or angular motion.
3. **Integrated Circuit:** This is usually a small potentiometer that controls the output voltage as it is turned, based on the desired position of the motor. The signals from the control system are interpreted and it adjusts the position of the motor accordingly.
4. **Motor:** The type of motor is not critical to a servo motor, and different types may be used. At the simplest, brushed permanent magnet DC motors are used, owing to their simplicity and low cost. Small industrial servo motors are typically electronically commutated brushless motors.
5. **Servo Case:** The purpose of this component is to protect and enclose the internal part of the servo motor. It typically provides structural support.

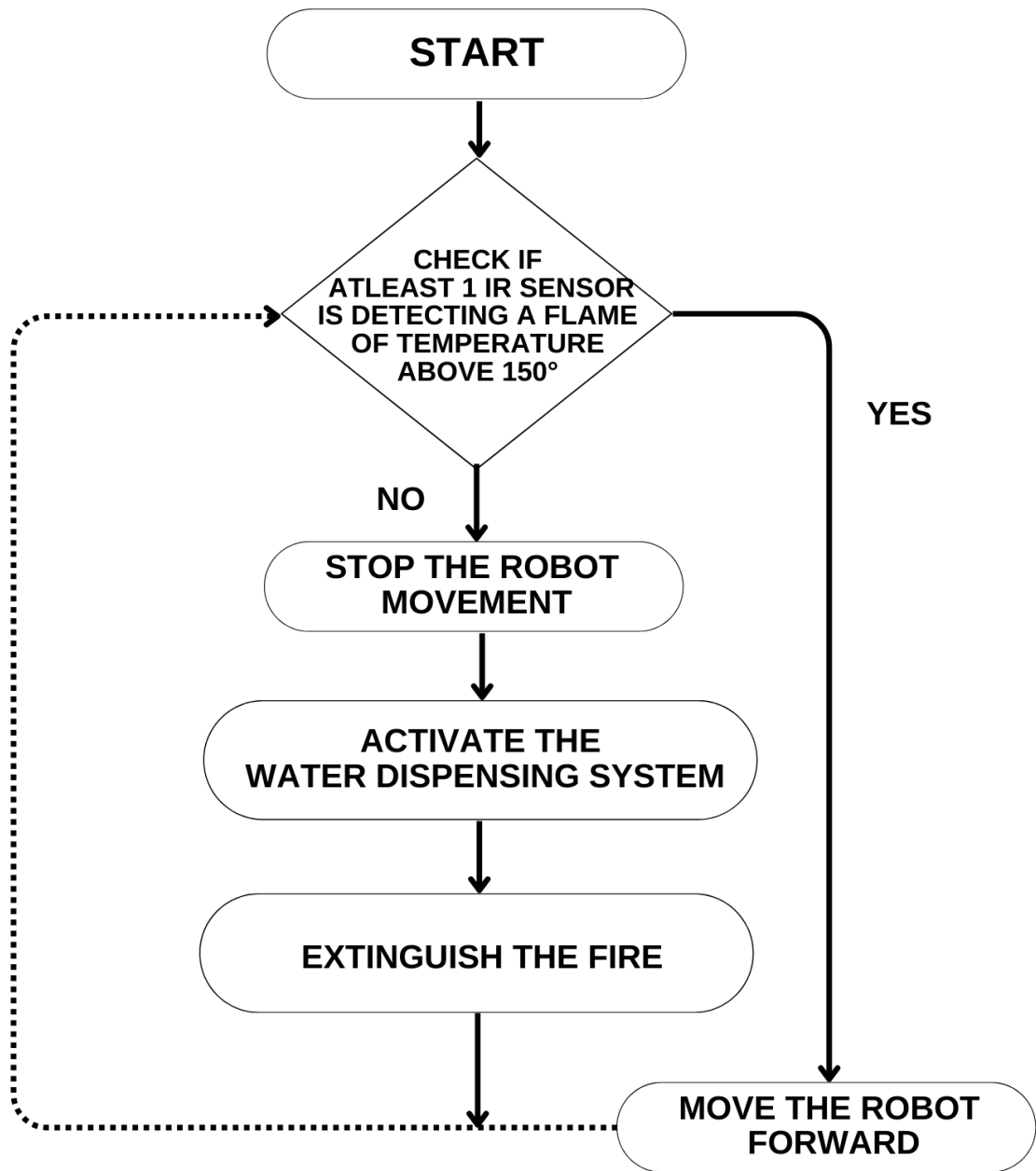
The servo motor retrieves control signals that specify its movement. The integrated circuit interprets these signals, adjusting voltage and current to the motor windings. Feedback that is taken from the potentiometer ensures accurate positioning of the servo motor. It ensures real-time information on the moto's actual position.

Circuit Diagram and Flow Chart

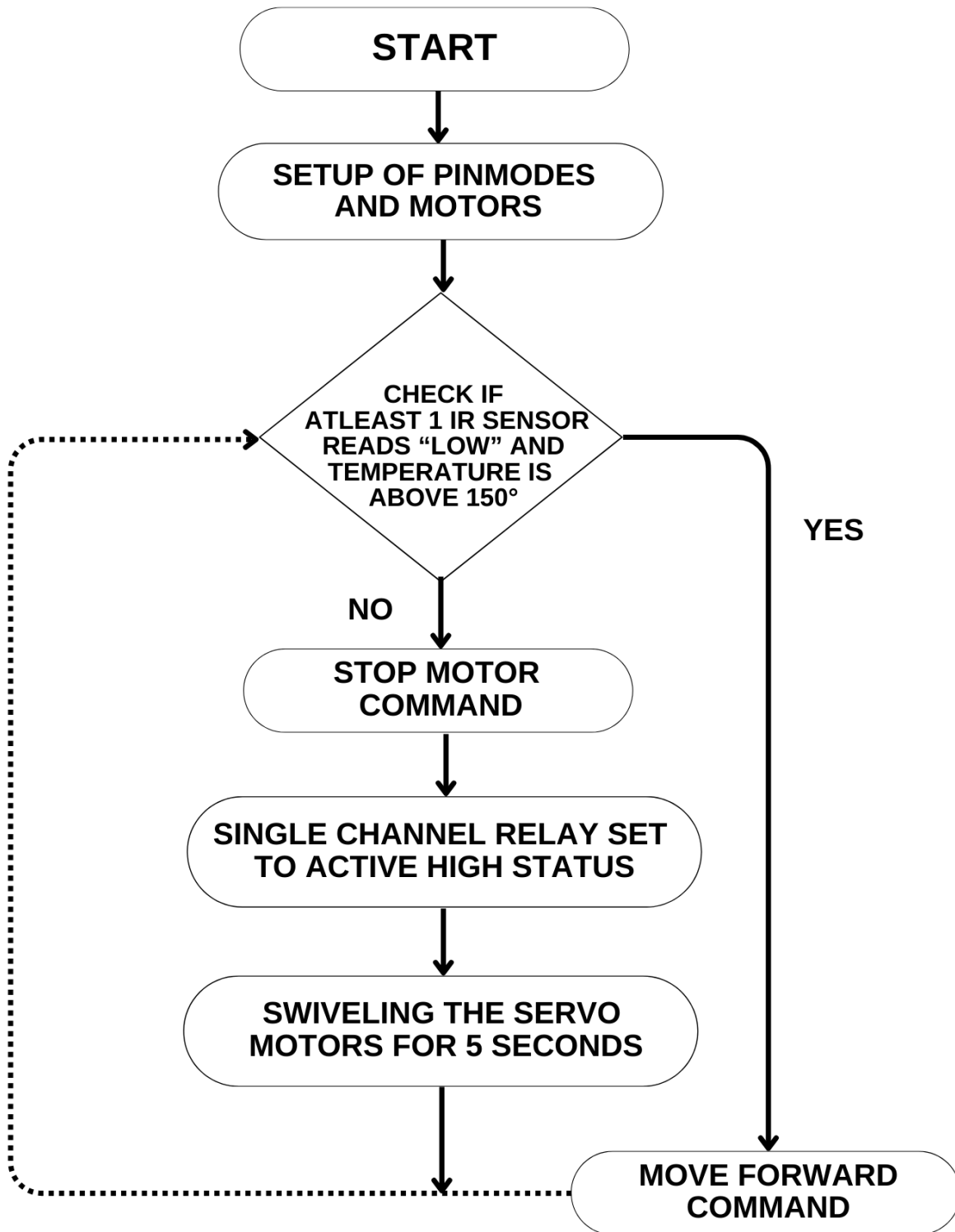
1. Schematic Diagram



2. Flowchart of Hardware Working



3. Flowchart of Code Working



Data Analysis and Acquisition

Table 1. Robot Response and Arduino Uno Action

Temperature	Flame IR Sensor 1	Flame IR Sensor 2	Flame IR Sensor 3	Robot Response	Arduino Uno Action
130.5	0	0	0	No action	No action
155.2	0	0	0	Activate fire extinguishing system	Activate pump
170.7	0	1	0	Activate fire extinguishing system	Activate pump
140.1	0	0	0	No action	No action
185.3	1	1	1	Activate fire extinguishing system	Activate pump
160.2	0	1	0	Activate fire extinguishing system	Activate pump
145.6	0	0	0	No action	No action
175.4	1	0	1	Activate fire extinguishing system	Activate pump
155.9	0	1	0	Activate fire extinguishing system	Activate pump
165.7	1	0	1	Activate fire extinguishing system	Activate pump

In the dataset above, the values in the column ‘Robot Response’ have been scaled using a scale factor where ‘No action’ is represented by 0, and ‘Activate fire extinguishing system’ is represented by 1. This dataset is to indicate which change in variables of flame IR and temperature corresponds to activation of the extinguishing system.

Table 2. Heat (Temperature) vs. Pump Status

Heat (Temperature)	Pump Status
120.5	0
175.3	1
90.7	0
200.1	1
140.2	0
160.8	1
130.6	0
180.4	1
110.9	0
165.7	1

The dataset above indicates the connection between the heat/temperature and pump status.

Table 3. Variation of Motor Speed over Time

Time	Motor Speed (rpm)	Wheel 1 Speed (rpm)	Wheel 2 Speed (rpm)	Wheel 3 Speed (rpm)	Wheel 4 Speed (rpm)	Direction
0	0	0	0	0	0	-
1	100	100	-100	100	-100	Forward
2	100	100	-100	100	-100	Forward
3	100	100	-100	100	-100	Forward
4	100	100	-100	100	-100	Forward
5	0	0	0	0	0	-

The above dataset indicates the motor speed (rpm) and individual wheel speed (rpm) over time. (negative speed here means moving in the backward direction).

Table 4. Temperature vs. Flame IR

Temperature	Flame IR Sensor 1	Flame IR Sensor 2	Flame IR Sensor 3
120.5	0	0	0
175.3	1	1	1
90.7	0	0	0
200.1	1	1	1
140.2	0	0	0
160.8	1	1	1
130.6	0	0	0
180.4	1	1	1
110.9	0	0	0
165.7	1	1	1

The above dataset indicates the relationship between the temperature and the readings of the Flame IR. Here we are able to find the correlation between the threshold value of temperature and the state of the Flame IR sensor.

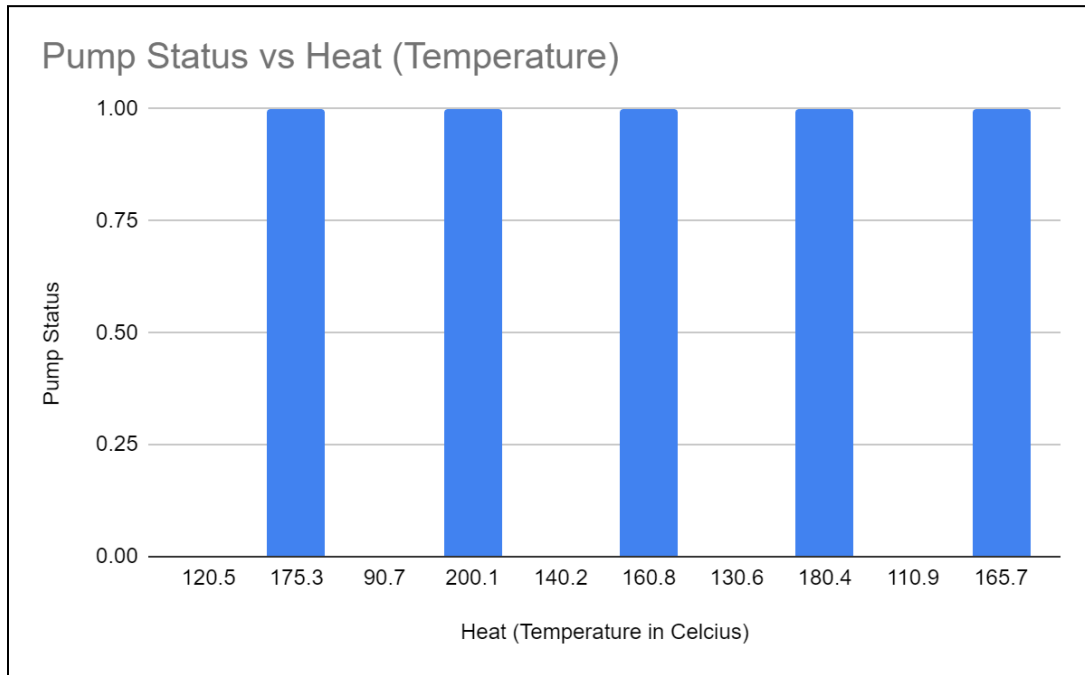
Table 5. Robot Movement vs. Flame IR

Robot Movement	Flame IR Sensor 1	Flame IR Sensor 2	Flame IR Sensor 3
1	0	0	0
0	1	0	0
0	1	1	0
0	1	0	0
0	1	1	1
0	1	1	0
0	1	0	0
0	1	1	1
0	1	1	0
0	1	1	1

The above dataset indicates the relationship between the robot movement and the state of the Flame IR sensors. The Flame IRs act as an OR gate describing the movement of the rob. If any one of the Flame IRs reads high it stops the movement of the robot else the robot continues to move forward.

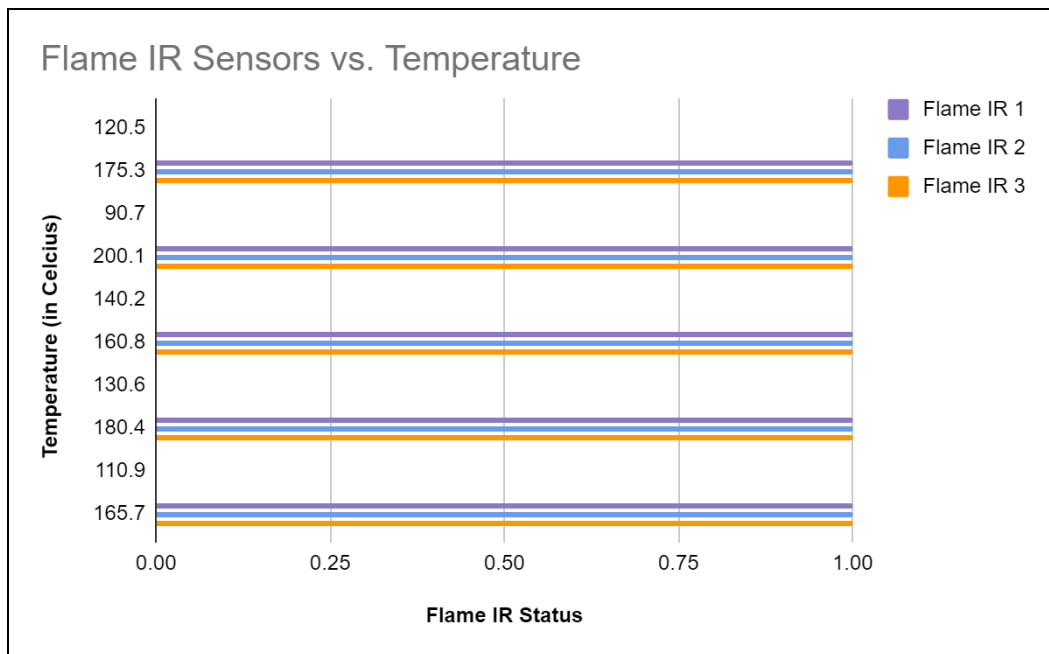
Data Visualisation

Graph 1. Pump Status vs. Heat (Temperature)

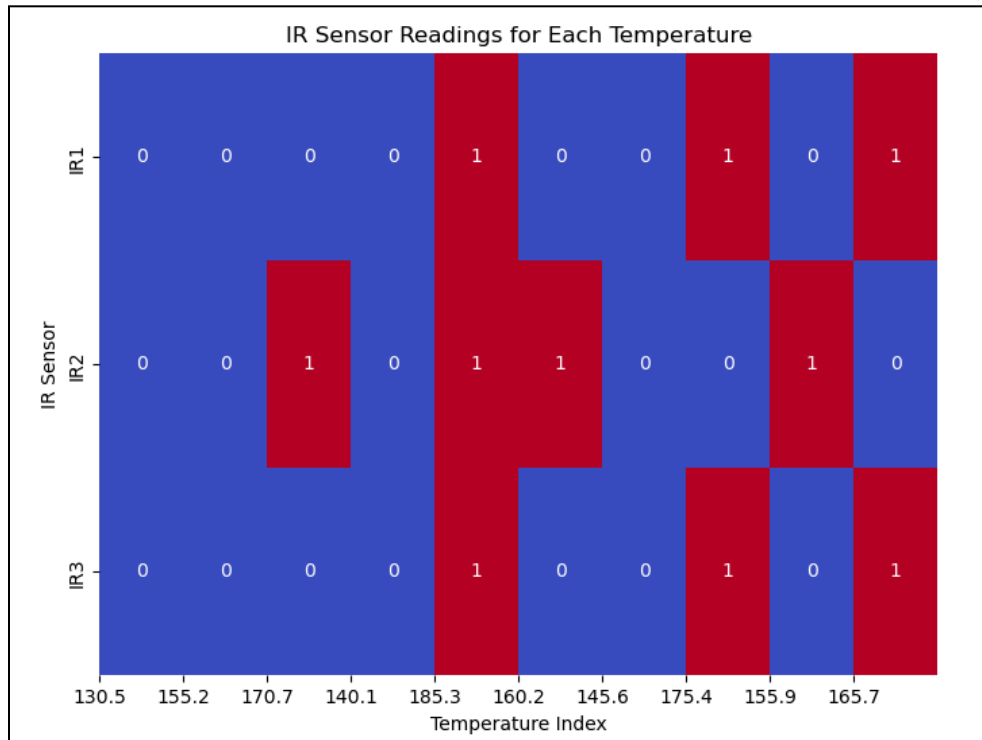


Graph 1 is a visualisation of Table 2. This graph represents the threshold value of temperature (150°C) above which the pump turns on.

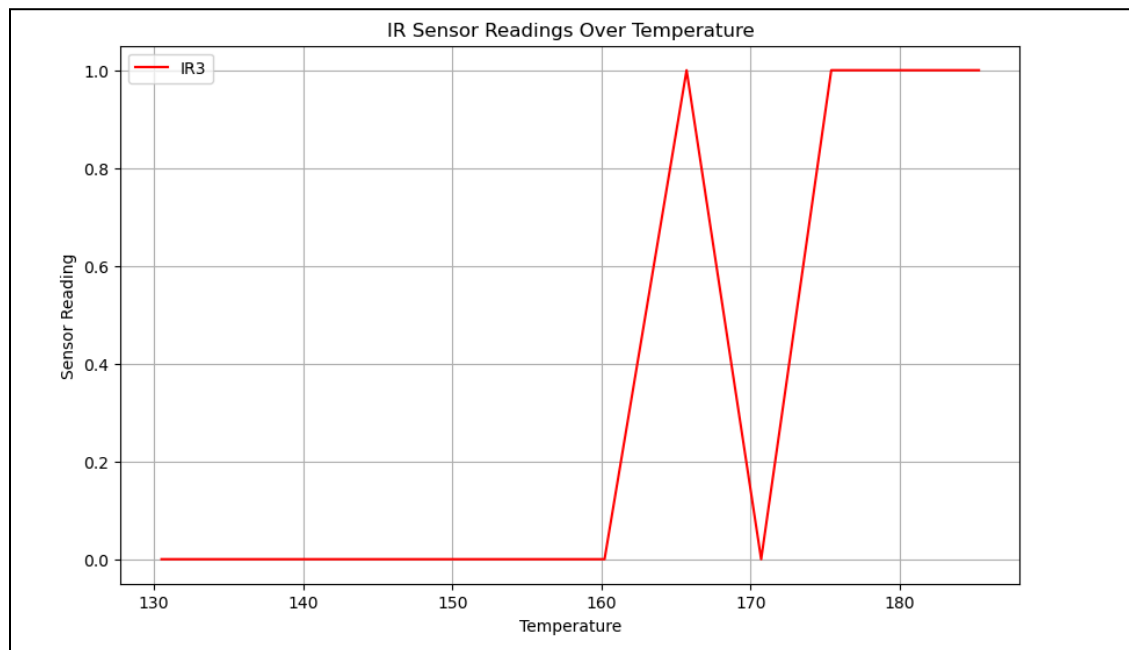
Graph 2. Flame IR Sensors vs. Temperature



Graph 3. IR Sensor Readings for Each Temperature

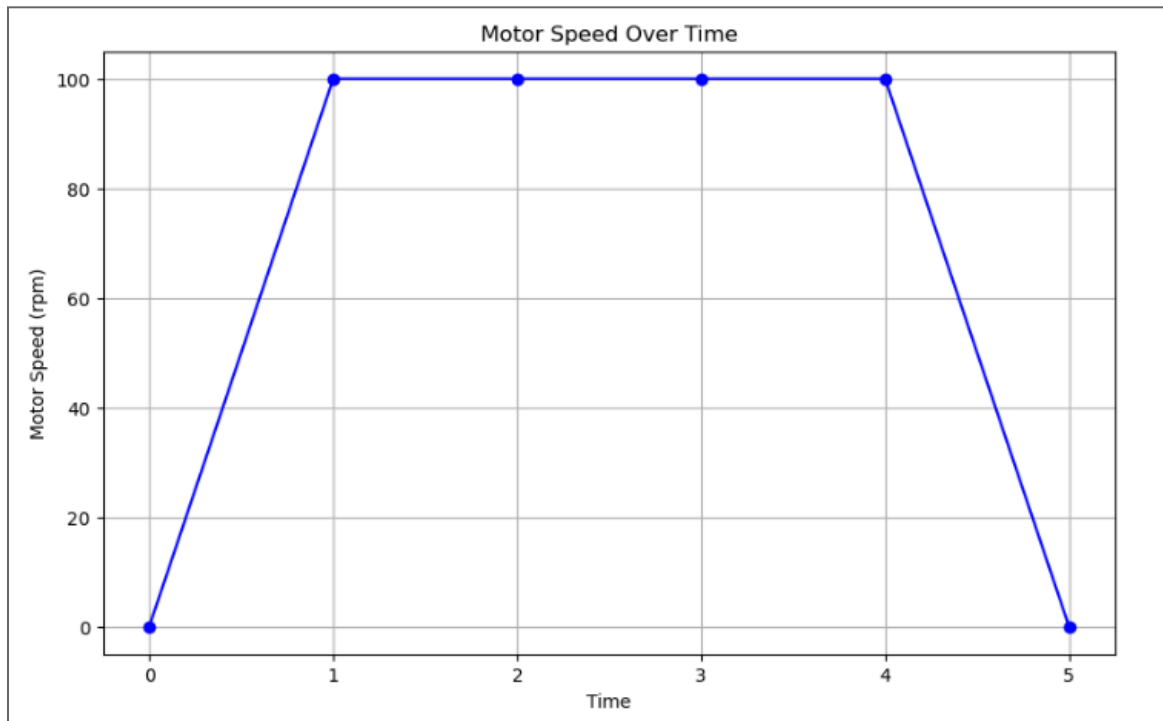


Graph 4. IR Sensor Readings Over Temperature

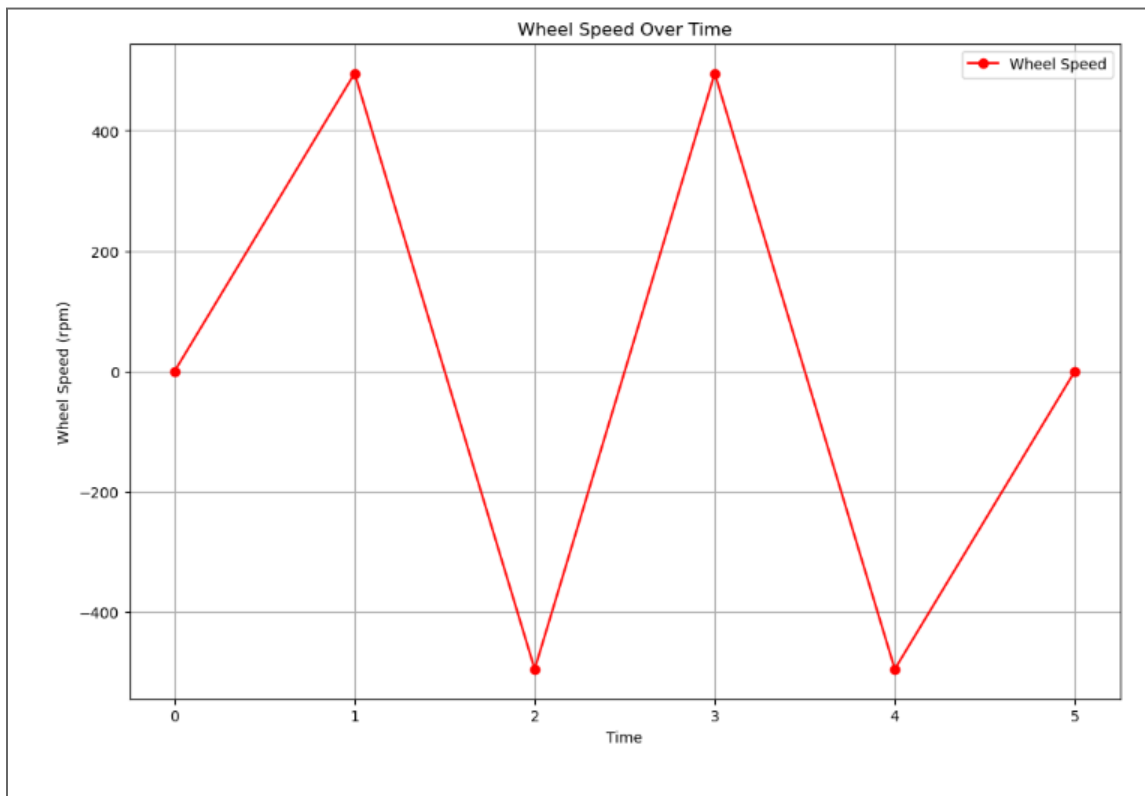


The graphs 2, 3 and 4 above are a visualisation of Table 3. Through these graphs, we can find the correlation between the threshold value of temperature and the state of the Flame IR sensor. Here we see above which set of temperatures results in a reading from the IRs. Graphs 2 and 3 show when the flame IR sensors read high on different temperatures and graph 4 represents when from which temperature onwards the spikes occur.

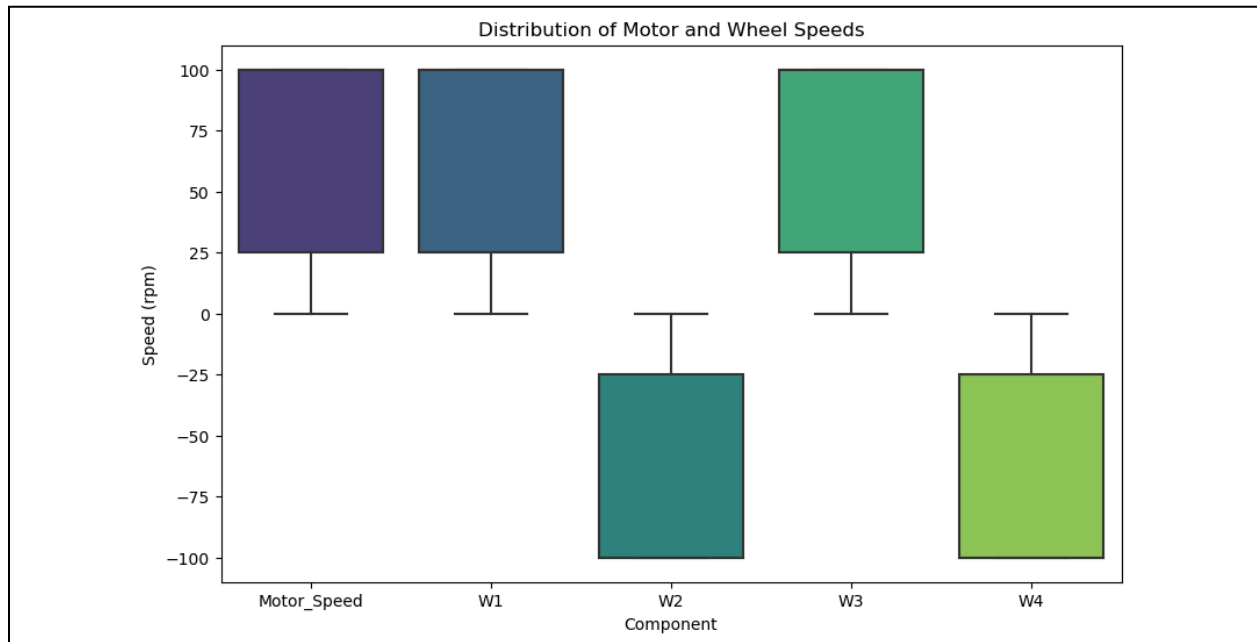
Graph 5. Motor Speed Over Time



Graph 6. Wheel Speed Over Time



Graph 7. Distribution of Motor and Wheel Speeds

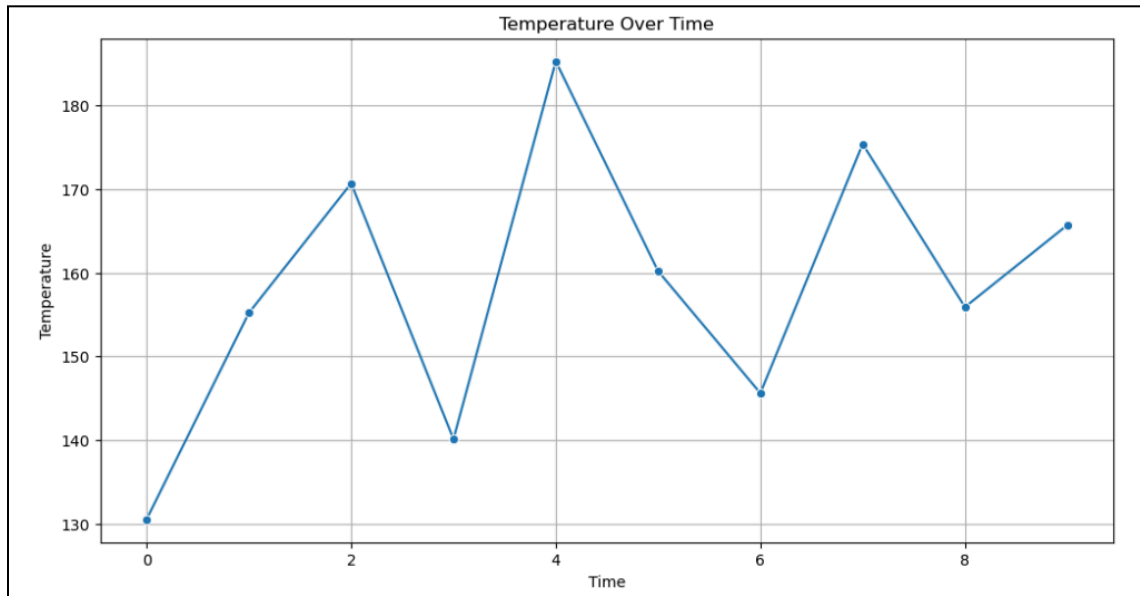


Graphs 5, 6, and 7 above are a visualisation of Table 4. The above graphs represent the polarities that each wheel has. In this case, wheel 2 and wheel 4 have a reverse polarity to wheel 1 and wheel 3 which is shown in graph 6 and 7.

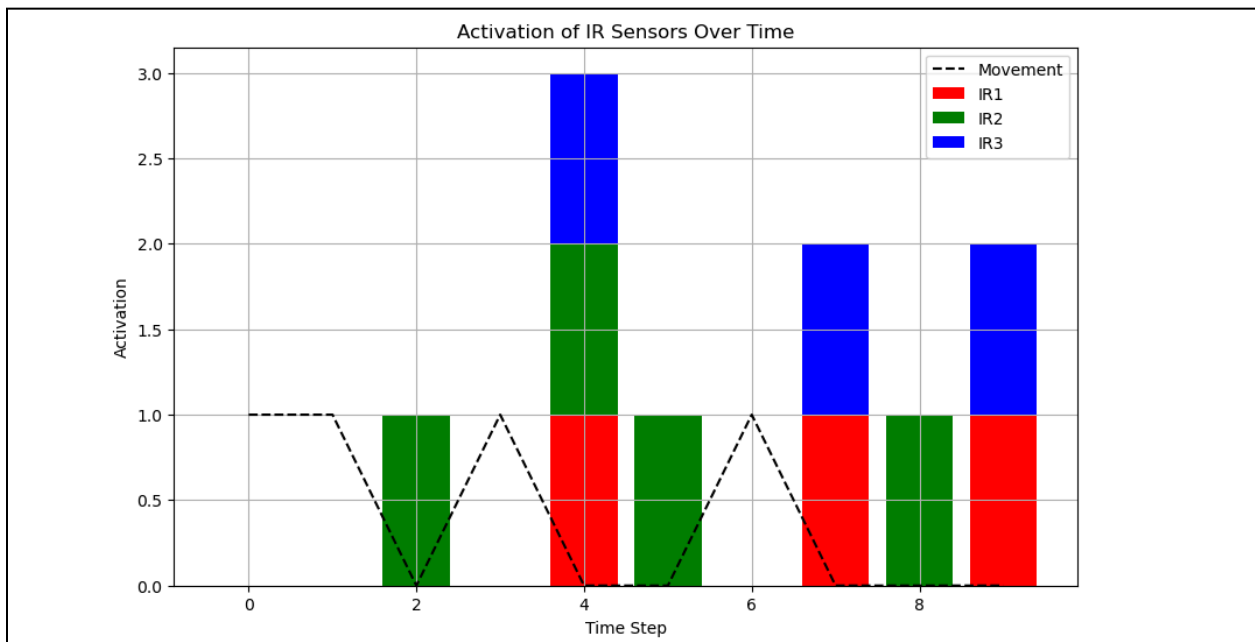
They also show that all the motors move in the same direction shown in graph 5.

Thus, for them to move in the same direction as wheel 1 and wheel 3 they must move in the negative direction (negative speed here means moving in the backward direction).

Graph 8. Temperature Over Time



Graph 9. Activation of IR Sensors Over Time



The above graphs 8 and 9 are concerning datasets 4 and 5. They represent the activation of Flame IR sensors and temperature readings over time. A relationship between the temperature exceeding the threshold value (150) and the flame sensor reading a high can be observed by comparing the two graphs parallelly. Here we see that spikes above 150 degrees in Graph 8 result in a high value of the IR sensor in Graph 9 and the movement becomes low.

Discussion and Conclusion

Problems faced

During the course of the project, many problems had been identified. The first had been understanding and implementing the connections and formulation of the hardware of the robot.

The second was that not enough power was being sent to the motors resulting in only 2 motors moving instead of all 4.

The third problem that was then encountered was the issue of the motors not working as intended with the code. After much debugging and hardware testing it was recognized that the power distribution to each motor was not equal and boiled down a faulty motor shield.

This then resulted in a sync error with the Arduino which prevented the code from being able to be uploaded into it.

```
Output
Sketch uses 3002 bytes (9%) of program storage space. Maximum is 32256 bytes.
Global variables use 261 bytes (12%) of dynamic memory, leaving 1787 bytes for local variables. Maximum is 2048 bytes.
avrdude: stk500_getsync() attempt 1 of 10: not in sync: resp=0x00
avrdude: stk500_getsync() attempt 2 of 10: not in sync: resp=0x00
avrdude: stk500_getsync() attempt 3 of 10: not in sync: resp=0x00
avrdude: stk500_getsync() attempt 4 of 10: not in sync: resp=0x00
avrdude: stk500_getsync() attempt 5 of 10: not in sync: resp=0x00
avrdude: stk500_getsync() attempt 6 of 10: not in sync: resp=0x00
avrdude: stk500_getsync() attempt 7 of 10: not in sync: resp=0x00
avrdude: stk500_getsync() attempt 8 of 10: not in sync: resp=0x00
avrdude: stk500_getsync() attempt 9 of 10: not in sync: resp=0x00
avrdude: stk500_getsync() attempt 10 of 10: not in sync: resp=0x00
Failed uploading: uploading error: exit status 1
```

After debugging and trying to slowly test out all possible points of failure. It pinned to the issue that the current Arduino in our possession was a Chinese replica. The issue was due to the fact that the clone does not use a standard USB chip instead uses a chip marked CH340. Thus the computer was not able to recognize it after many uploads.

This faulty Arduino also caused our Flame IRs to continuously be set to high irrespective of whether a fire was detected or not.

Solutions

To overcome the first problem. By building an understanding through research and through trial and error we were able to understand the use of each port and how they are communicating with each other. This allowed us to build our hardware.

The second problem after some research was since we had been using alkaline batteries, while the voltage rating was more than enough to run the motors alkaline batteries tend to have a low current output, and as the motors on the shield are connected in parallel the current being sent mattered the most. Thus by swapping to lithium ions with sufficient current, we resolved this issue.

The solution to the 3rd issue was simply swapping out the replica Arduino and motor shield for new ones that had the proper hardware. These ended up resolving all issues with the motors along with the flame IR sensor issues.

Application

This project is applicable to many different fields such as firefighting, search and rescue, industrial safety and remote monitoring. In addition to reducing firefighter safety risks, the fire extinguishing robot enhances emergency response capabilities. Search and rescue missions, industrial hazard monitoring and disaster relief efforts can all benefit from the versatility of the robot as it can help navigate hazardous environments and prevent fire-related accidents. Additionally, this robot can be used for remote monitoring applications such as detecting and extinguishing forest fires.

Future work

In this project, there are numerous enhancements that can be made to improve the ability of firefighters. This is done by using modern sensors, which improve the systems' capability for detecting fire and maintaining situational consciousness. By integrating AI into it, predictive analytics and real-time optimization of firefighting strategies become achievable.

Operation flexibility is enhanced by enabling remote monitoring and control. The process mobility has a need for further improvement while at the same time enhancing its fire extinction capacity, efficiency as well as exploring other possibilities for expansion such as industrial kitchens, restaurant safety, military deployment etc.

Future Scope

We intend on enhancing the capabilities of our robot by controlling it remotely. By setting up communication protocols like Bluetooth and Wifi and control interfaces can be used to enable the users to command the robot, the control algorithms interpret commands. Additionally, we want to integrate a camera system for live video surveillance. We would have to develop image processing algorithms and protocols for live video streams. We also want to improve the weight capacity of the robot. We can strengthen the robot's chassis and replace and upgrade existing components.

Advantages

It excels accurately in accurately pointing to the direction of the fire sources. Its sensing capabilities are finely tuned for precision with increased feasibility. The robot effectively reduces the need for human intervention. It is very reliable and cost-effectiveness making it a dependable performance without straining budgets. It does not act differently to changes in weather conditions.

Disadvantages

Firstly, we don't have the monitoring system for the vehicle, this way we don't have the ability to track its movements in real-time. The lack of remote control for the navigation restricts the ability of our robot to navigate efficiently. Our robot is suitable for applications weighing less than 3.5 Kg. It is also important to note that our robot is not designed for extinguishing large fires, its capabilities are tailored for smaller-scale applications.

References

1. Delia James, Anusha Wilson (17th December, 2015): *Automatic Fire Fighting Robot*
<https://www.slideshare.net/slideshow/fire-fighting-robot-ppt-56227281/56227281>
2. K Shamili, Devi, K Akhileswar ,C H Vinayaka (July , 2020): *Fire Fighting Robot*
https://www.researchgate.net/publication/348929351_FIRE_FIGHTING_ROBOT
3. Kondeti Chirunadh, Mr. G. Sai Varaprasad Goud, Miss. G. Tejaswini, Mr. B. Srinivasa Rao (6th June, 2023) : *Arduino Based Fire Fighting Robot*
https://www.irjmets.com/uploadedfiles/paper/issue_6_june_2023/42309/final/fin_irjmets_1687229062.pdf
4. Mohd Shahid (30th December, 2022) : *Fire Fighter Robot using Arduino | Fire fighting Robot*
https://techatronic.com/fire-fighter-robot-using-arduino-fire-fighting-robot/#google_vignette
5. Pooja Krishna More (20th August, 2020) : *Fire-fighting final report*
<https://www.slideshare.net/PoojaMore61/fire-fighting-final-report>
6. Teh Nam Khoon (2012): *Fire Fighting Robot, Dissertation*
https://utpedia.utp.edu.my/id/eprint/4025/1/Dissertation_Fire_Fighting_Robot_12479.pdf

Appendix

Git-Hub:

<https://github.com/ArjunSyam/Fire-Extinguishing-Robot>

Working Model Videos:

https://drive.google.com/drive/folders/1o-TIIQPmIckoMIg9x1_CgbRYPQE-ald2?usp=drive_link

Code Snippets:

```
#include <AFMotor.h>
#include <Servo.h>

AF_DCMotor motor1(1, MOTOR12_1KHZ);
AF_DCMotor motor2(2, MOTOR12_1KHZ);
AF_DCMotor motor3(3, MOTOR34_1KHZ);
AF_DCMotor motor4(4, MOTOR34_1KHZ);

const int maxSpeed = 100;    // Maximum speed for the motors
const int acceleration = 5;  // Acceleration rate for smooth speed changes
const int relay = 1;
const int flamePin1 = 9;    // Digital pins for flame sensors
const int flamePin2 = 11;
const int flamePin3 = 13;
const int tempsensor = 'A0';
float temperature;

Servo myServo;              // Create a Servo object
int servoPin = 10;          // Pin for servo motor

void setup() {
    Serial.begin(9600);

    pinMode(flamePin1, INPUT_PULLUP); // Set flame sensor pins as inputs with
    // internal pull-up resistors
    pinMode(flamePin2, INPUT_PULLUP);
    pinMode(flamePin3, INPUT_PULLUP);
```

```

pinMode(relay, OUTPUT);

pinMode(tempsensor, INPUT);

motor1.setSpeed(maxSpeed);
motor2.setSpeed(maxSpeed);
motor3.setSpeed(maxSpeed);
motor4.setSpeed(maxSpeed);
motor1.run(RELEASE);
motor2.run(RELEASE);
motor3.run(RELEASE);
motor4.run(RELEASE);

myServo.attach(servoPin); // Attach servo to pin 10
myServo.write(90);        // Set initial servo position to 90 degrees
}

```

Here we define the libraries `<AFMotor.h>` and `<Servo.h>` to allow us to access functions to control the motors and servos.

We set the pins and PWM rate for all the 4 motors. We then set the max speed, acceleration, and pins for the IR sensors, servo motor, temperature sensor, and relay. Using these pins we define the pin mode for each component in the setup and initialise the movement of the motors.

```

void loop() {
  if (!flameDetected()) {
    moveForward();
    delay(2000);
  } else {
    stop();
    delay(500);
    digitalWrite(relay, HIGH);
    delay(50);
    swiveServo();
    delay(3000);
    digitalWrite(relay, LOW);
  }
}

```

Within the loop we check if the function `flameDetected()` returns a false value. If it does then the robot moves forward with a delay of 2 secs.

If the function returns true then we first stop the motor's movement. Then after 500 milliseconds change relay input to high. Then we run the `swiveServo()` command And then after 3 secs change the relay input back to low.

```
void moveForward() {
    Serial.println("Moving forward\n");
    accelerateMotors(motor1, motor2, motor3, motor4, maxSpeed);
}

void stop() {
    Serial.println("Stopped\n");
    decelerateMotors(motor1, motor2, motor3, motor4);
    motor1.run(RELEASE);
    motor2.run(RELEASE);
    motor3.run(RELEASE);
    motor4.run(RELEASE);
}

void accelerateMotors(AF_DCMotor& motor1, AF_DCMotor& motor2, AF_DCMotor& motor3,
AF_DCMotor& motor4, int targetSpeed) {
    motor1.run(FORWARD);
    motor2.run(BACKWARD);
    motor3.run(FORWARD);
    motor4.run(BACKWARD);
    for (int speed = 0; speed <= targetSpeed; speed += acceleration) {
        motor1.setSpeed(speed);
        motor2.setSpeed(speed);
        motor3.setSpeed(speed);
        motor4.setSpeed(speed);
        delay(10);
    }
}

void decelerateMotors(AF_DCMotor& motor1, AF_DCMotor& motor2, AF_DCMotor& motor3,
AF_DCMotor& motor4) {
    int currentSpeed = 100;
    while (currentSpeed > 0) {
```

```

    motor1.setSpeed(currentSpeed);
    motor2.setSpeed(currentSpeed);
    motor3.setSpeed(currentSpeed);
    motor4.setSpeed(currentSpeed);
    currentSpeed -= acceleration;
    delay(10);
}
}

```

Here these functions are used to control the movement of the motors. Function `stop()` first decelerates the motors from current speed to zero using function `decelerateMotors` and then releases power for motors using `motor1.run(RELEASE)`.

The function `moveForward()` first accelerates the motors from 0 speed to 100 using the function `accelerateMotors` and then moves the motors forward. Motor2 and motor4 are set backward as they have reverse polarity.

```

void swiveServo() {
    for (int angle = 90; angle >= 0; angle -= 5) {
        myServo.write(angle);
        delay(50);
    }
    for (int angle = 0; angle <= 180; angle += 5) {
        myServo.write(angle);
        delay(50);
    }
    for (int angle = 180; angle >= 90; angle -= 5) {
        myServo.write(angle);
        delay(50);
    }
}

```

For the first loop rotates the servo from 90 degrees to 0, decreasing the angle by 5 each 50 milliseconds. The next loop rotates the servo from 0 degrees to 180 degrees and the final one rotates from 180 to 90.